

The Fast Track

JavaScript · CSS · HTML → TypeScript → React → Next.js

A 7-Week Curriculum for the Experienced Developer

Designed for: 25-year coding veteran returning to modern frontend

Daily commitment: ~2 hours focused study + practice

Total duration: 7 weeks

Course Map

1. **Materials & Setup:** Books, links, tools, and accounts needed before Day 1
2. **Weeks 1-2:** Modern JavaScript, HTML5 & CSS3 fundamentals
3. **Week 3:** TypeScript
4. **Weeks 4-5:** React (with hooks and modern patterns)
5. **Weeks 6-7:** Next.js (App Router, full-stack)
6. **Appendix:** Reference tables, ecosystem, and what to skip

Materials & Setup

Books, free resources, tools, and accounts to gather before Day 1.

Books — what to buy or read free

You don't need to buy all of these. Bare minimum: *Exploring JavaScript* (free), the official docs (free), and one solid TypeScript book if you want a paper reference. Everything else is optional. I've marked them honestly.

JavaScript

Exploring JavaScript (ES2025 edition) Free online

Dr. Axel Rauschmayer

The single best reference for modern JS for someone with programming experience. Free to read at exploringjs.com/js. Paid version (~\$39, DRM-free PDF/EPUB) includes exercises and flashcards — worth it if you want offline study materials.

You Don't Know JS Yet (2nd edition) Free online

Kyle Simpson

Six short books, all free on GitHub: github.com/getify/You-Dont-Know-JS. For this curriculum, only read "Scope & Closures" and "Async & Performance". Read the rest later if you want deep understanding.

Deep JavaScript: Theory and techniques Free online Optional

Dr. Axel Rauschmayer

Companion to *Exploring JavaScript*. Deeper material — read after you're comfortable. Free at exploringjs.com/deep-js.

HTML & CSS

MDN Web Docs Free

Mozilla

Not a book but the canonical reference. [HTML reference](#) and [CSS reference](#) are better than any HTML/CSS book published in the last 5 years.

Learn CSS Free

Google / web.dev (Andy Bell, Rachel Andrew, Una Kravets, Adam Argyle)

A structured, modern course written by people who literally helped design the CSS specs. web.dev/learn/css. This is your Week 2 backbone.

Learn HTML Free

Google / web.dev

Companion to Learn CSS. web.dev/learn/html. Skim — most of it is review for you.

CSS for JavaScript Developers \$\$ Optional

Josh Comeau

Paid course (~\$300, but goes on sale). Best CSS course in existence for developers who already code. If budget allows, buy it instead of Week 2's free path. css-for-js.dev. His free blog at joshwcomeau.com has many of the same insights free.

TypeScript

Exploring TypeScript (TS 5.8 edition) Free online

Dr. Axel Rauschmayer

Same author as *Exploring JavaScript*, same style. Free at exploringjs.com/ts. Use alongside the official handbook.

TypeScript Handbook Free

Microsoft (official)

The official docs are genuinely excellent. typescriptlang.org/docs/handbook. Primary material for Week 3.

Effective TypeScript: 83 Specific Ways to Improve Your TypeScript (2nd ed.) \$\$

Dan Vanderkam · O'Reilly, 2024 · ISBN 978-1098155063

The *Effective C++/Effective Java*-style book for TypeScript. Updated for TS 5. If you buy one TS book, buy this one. ~400 pages of "do this, not that" — exactly the format that suits experienced devs.

React

Official React Documentation Free

Meta (the React team)

The new docs (launched 2023) are a full tutorial. react.dev/learn. This is your Week 4 primary material. No book is better.

Patterns.dev Free online

Lydia Hallie & Addy Osmani

Modern design patterns for React/Next.js. Free at patterns.dev. Read after Week 4.

Total TypeScript / Epic React \$\$\$ Optional

Matt Pocock / Kent C. Dodds

Both excellent paid courses (totaltypescript.com, epicreact.dev). Consider after the curriculum if you want to deepen further. Matt Pocock also publishes a huge amount of free TS content on YouTube and Twitter.

Next.js

Next.js Learn (official course) Free

Vercel

A guided tutorial that builds a real app. nextjs.org/learn. Your Week 6 primary material.

Next.js Documentation Free

Vercel

nextjs.org/docs. Use as reference throughout Weeks 6–7. The App Router section is what matters.

Software & accounts to set up before Day 1

- ☐ Install **Node.js LTS** (v20 or later) from nodejs.org
- ☐ Install **pnpm**: `npm install -g pnpm` (faster, disk-efficient package manager)
- ☐ Install **VS Code** from code.visualstudio.com
- ☐ Install VS Code extensions: **ESLint**, **Prettier**, **ES7+ React/Redux/React-Native snippets**, **Tailwind CSS IntelliSense**, **Error Lens**, **GitLens**
- ☐ Install a modern browser: **Chrome** or **Firefox Developer Edition** (the DevTools matter)
- ☐ Sign up for free accounts: [GitHub](https://github.com) (host your work), [Vercel](https://vercel.com) (deploy Next.js apps), [Neon](https://neon.tech) or [Supabase](https://supabase.com) (free Postgres for Week 6)
- ☐ Bookmark in your browser: [MDN](https://mdn.com), caniuse.com, react.dev, [Next.js docs](https://nextjs.org/docs)
- ☐ Create a folder `~/fast-track/` with subfolders `week-01` through `week-07`

Interactive practice (free, do these as scheduled)

- [Flexbox Froggy](#) — scheduled for Day 8. Finish in one sitting.
- [Grid Garden](#) — scheduled for Day 9. Same.
- [TypeScript Exercises](#) — supplement during Week 3.
- [Matt Pocock's Beginner's TypeScript](#) — free, supplement for Week 3.
- [React patterns & exercises](#) — supplement Week 4-5.

YouTube essentials (free)

- ["In The Loop" by Jake Archibald](#) — 35 min. Scheduled for Day 4. The single best explanation of the event loop in existence.
- [Matt Pocock's channel](#) — short, dense TypeScript clips. Watch through Week 3.
- [Theo \(t3.gg\)](#) — opinionated, current commentary on the React/Next.js ecosystem.
- [Jack Herrington](#) — deep dives on React, Next.js patterns.

Newsletters worth subscribing to (free)

- [JavaScript Weekly](#) — Friday digest of JS news. Light skim, high signal.
- [React Status](#) — weekly React/Next.js news.
- [Frontend Focus](#) — CSS, HTML, browser feature news.
- [Bytes by ui.dev](#) — funniest JS newsletter, decent signal.

Realistic budget

Tier	What you buy	Approx. cost
Zero-cost	Everything free above. Sufficient to complete the curriculum.	\$0
Recommended	<i>Exploring JavaScript</i> paid bundle + <i>Effective TypeScript</i>	~\$40 + ~\$40
Premium	Above + Josh Comeau's <i>CSS for JS Devs</i> + Matt Pocock's <i>Total TypeScript</i>	~\$700+

For your stated goal — get fluent and ship — the **recommended tier (~\$80)** is the sweet spot. Don't fall into the paid-course trap of buying material faster than you consume it.

How to Use This Material

Each week has 5 working days plus 2 weekend "consolidation" days for the capstone project. Skip the weekend if you've internalized the material — these books are scoped for someone who codes fast and reads faster.

The non-negotiable rule: you build something every single day. Reading without typing is the failure mode for experienced devs — you will recognize concepts and assume you know them. You don't, until your fingers do.

Setup before Day 1

Install Node.js LTS, VS Code (extensions: ESLint, Prettier, ES7 React snippets), and a modern browser. Open a terminal and run `node --version` — confirm you're on v20+. Initialize a folder `~/fast-track` with subfolders for each week.

Week 1 — Modern JavaScript Core

Goal: Be fluent in ES2015–ES2024 syntax, the module system, and the async model.

Day 1 — Variables, Scope, and the Module System

Reading (45 min): [Exploring JS — Variables and assignment](#), [Modules](#).

Key concepts:

- `let` and `const` have block scope; `var` has function scope. Forget `var` exists.
- The Temporal Dead Zone: accessing a `let` / `const` before declaration throws.
- ES modules use `import` / `export`. They are strict-mode by default and statically analyzed.
- Files must be `.mjs` or have `"type": "module"` in `package.json`.

```
// math.js
export const PI = 3.14159;
export function area(r) { return PI * r * r; }
export default function circumference(r) { return 2 * PI * r; }

// app.js
import circumference, { PI, area } from './math.js';
console.log(area(5));
```

Exercise: Split a single-file Node script (any small project — a calculator, a fizzbuzz, anything) into 3+ modules with both named and default exports.

Day 2 — Functions, Arrow Functions, Destructuring, Spread

Reading (45 min): [Exploring JS — Callables](#), [Destructuring](#).

Key concepts:

- Arrow functions: lexical `this` (inherit from enclosing scope), no `arguments` object, can't be used as constructors.
- Default parameters, rest parameters (`...args`), spread operator (`...arr`).
- Object/array destructuring with renaming, defaults, and nesting.

```
// Destructuring with defaults and renaming
const { name: userName = 'Guest', age } = user;

// Rest and spread
function sum(...nums) { return nums.reduce((a, b) => a + b, 0); }
const merged = { ...defaults, ...overrides };
const newArr = [...oldArr, newItem];

// Arrow function 'this' behavior
class Timer {
  constructor() { this.seconds = 0; }
  start() {
    setInterval(() => { this.seconds++; }, 1000); // arrow keeps `this`
  }
}
```

Exercise: Write a function that takes a deeply nested user object and returns a flattened summary using only destructuring (no dot access). Then refactor a small class using arrow methods.

Day 3 — Promises and Async/Await

Reading (60 min): [Exploring JS — Promises, Async functions](#). This is the most important day of Week 1.

Key concepts:

- A Promise is a placeholder for a future value with three states: pending, fulfilled, rejected.
- `async` functions always return a Promise. `await` pauses execution until the Promise settles.
- `Promise.all`, `Promise.allSettled`, `Promise.race`, `Promise.any` — know what each does.
- Always handle rejections. An unhandled rejection crashes Node processes by default.

```
// Sequential vs parallel
async function sequential() {
  const a = await fetchA(); // waits
  const b = await fetchB(); // then waits again
  return [a, b];
}

async function parallel() {
  const [a, b] = await Promise.all([fetchA(), fetchB()]); // both at once
  return [a, b];
}

// Error handling
async function safe() {
  try {
    const data = await fetch('/api').then(r => r.json());
    return data;
  } catch (err) {
    console.error('Failed:', err);
    return null;
  }
}
```

Exercise: Build a script that fetches data from 3 public APIs (e.g., GitHub, JSONPlaceholder, public weather) in parallel, combines results, and handles partial failures gracefully.

Day 4 — The Event Loop, Microtasks, and "Why is my code weird"

Reading (60 min): [Exploring JS — Foundations of async programming](#), then watch [Jake Archibald's "In The Loop"](#) (free on YouTube — 35 min, the single best explanation in existence).

Key concepts:

- JavaScript is single-threaded. The runtime has a call stack, a task queue (macrotasks), and a microtask queue.
- Microtasks (Promise callbacks, `queueMicrotask`) run before the next macrotask.
- `setTimeout(fn, 0)` is not immediate — it waits for the call stack to empty.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1, 4, 3, 2
// Reason: sync first, then microtasks (Promise), then macrotasks (setTimeout)
```

Exercise: Predict the output of 10 mixed sync/async snippets before running them. Get them all right. This is the muscle that pays off in React's `useEffect` later.

Day 5 — Iterators, Generators, Classes, Prototypes

Reading (60 min): [Exploring JS — Synchronous iteration](#), [Classes](#), [Prototype chains](#).

Key concepts:

- An iterable has a `[Symbol.iterator]()` method. `for...of` uses this protocol.
- Generators (`function*`) produce iterables lazily with `yield` .
- JS classes are syntactic sugar over prototypes. Understanding the prototype chain matters when debugging.
- Private fields with `#` prefix (ES2022).

```
// Generator: infinite lazy sequence
function* naturals() {
  let n = 1;
  while (true) yield n++;
}
const gen = naturals();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2

// Class with private field
class BankAccount {
  #balance = 0;
  deposit(amount) { this.#balance += amount; return this; }
  get balance() { return this.#balance; }
}
```

Exercise: Write a generator that paginates through a paginated API one page at a time. Build a class hierarchy with private state for a small simulation (e.g., bank, inventory, game).

Weekend 1 capstone

Build a Node CLI tool: fetches data from a public API, transforms it, and writes a report file. Use ES modules, `async/await`, classes, and at least one generator. ~150–200 lines. Push it to GitHub. This is your proof-of-fluency artifact.

Week 2 — HTML5 + Modern CSS

Goal: Write semantic, accessible HTML and modern CSS (Grid, Flexbox, custom properties, container queries) without reaching for a framework.

Day 6 — Semantic HTML and Accessibility

Reading (90 min): [MDN HTML element reference](#) (skim), [MDN Accessibility "Learn" path](#) (read).

Key concepts:

- Use semantic tags: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`. Screen readers and search engines depend on them.
- Forms: every `<input>` needs a `<label>`. Use the right `type` (email, tel, number, date).
- ARIA attributes only when there's no native HTML equivalent. The rule: "no ARIA is better than bad ARIA."
- Newer interactive elements: `<dialog>` (with `showModal()`), `<details>` / `<summary>`.

```
<main>
  <article aria-labelledby="post-title">
    <h1 id="post-title">Article Title</h1>
    <p>Content...</p>
  </article>

  <dialog id="confirm">
    <form method="dialog">
      <p>Are you sure?</p>
      <button value="cancel">Cancel</button>
      <button value="ok">OK</button>
    </form>
  </dialog>
</main>
```

Exercise: Take any landing page screenshot. Write the pure semantic HTML for it — no CSS yet. Test it with a screen reader (macOS VoiceOver: Cmd+F5; Windows: NVDA, free). You'll be humbled.

Day 7 — The Box Model, Selectors, Custom Properties

Reading (60 min): [web.dev Learn CSS](#) — modules 1–6 (Box model, Selectors, The cascade, Specificity, Inheritance, Color).

Key concepts:

- **box-sizing: border-box** on everything. Add it once globally.
- Specificity: inline > ID > class/attr/pseudo-class > element. **:where()** has zero specificity — use it to keep things low.
- New selectors: **:has()** (parent selector!), **:is()**, **:where()**.
- Custom properties (**--color-primary**) are runtime-evaluated and inheritable. They're the foundation of theming.

```
:root {
  --color-primary: #635bff;
  --color-bg: #fff;
  --space-1: 0.5rem;
  --space-2: 1rem;
}

[data-theme="dark"] {
  --color-bg: #0a0a0a;
}

*, ::before, ::after { box-sizing: border-box; }

button {
  background: var(--color-primary);
  padding: var(--space-1) var(--space-2);
}

/* :has() — style a card differently if it contains an image */
.card:has(img) { padding: 0; }
```

Exercise: Build a button component with 4 variants (primary, secondary, ghost, danger) using only custom properties. Add a dark mode toggle that swaps a single **data-theme** attribute on **html**.

Day 8 — Flexbox

Reading (45 min): [CSS Tricks — Complete Guide to Flexbox](#) — the canonical reference.

Practice (60 min): Play [Flexbox Froggy](#) end-to-end. Don't skip it because it looks like a game — it's the fastest way to internalize the model.

Key concepts:

- Flexbox is 1D — items flow in one direction (row or column).
- **justify-content** aligns on the *main* axis; **align-items** on the *cross* axis.
- **flex: 1** is shorthand for **flex: 1 1 0%** — grow, shrink, basis.
- **gap** works in flex now — stop using margins for spacing between items.

```
.toolbar {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.toolbar .spacer { flex: 1; } /* pushes everything after it to the right */
```

Exercise: Build a navbar, a card with a footer pinned to the bottom, and a centered modal — all with flexbox.

Day 9 — Grid

Reading (60 min): [CSS Tricks — Complete Guide to Grid.](#)

Practice: Play [Grid Garden](#). Then read [Rachel Andrew's posts on Grid](#) (she literally co-designed the spec).

Key concepts:

- Grid is 2D — rows and columns simultaneously.
- `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))` gives you a responsive grid in one line with no media queries.
- Named grid areas make complex layouts readable.
- `subgrid` lets nested grids inherit tracks from the parent.

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr 200px;
  grid-template-areas:
    "header header header"
    "nav   main   aside"
    "footer footer footer";
  gap: 1rem;
}
.layout > header { grid-area: header; }
.layout > nav    { grid-area: nav; }
.layout > main   { grid-area: main; }

/* Responsive card grid, no media queries needed */
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1rem;
}
```

Exercise: Rebuild the layout of a site you like (Stripe, Linear, Vercel) using Grid for page structure and Flexbox for component internals.

Day 10 — Container Queries, Cascade Layers, Modern Tricks

Reading (60 min): [web.dev — Container queries](#), [MDN — @layer](#), [CSS nesting](#).

Key concepts:

- Container queries let a component respond to its *container's* size, not the viewport. Game-changer for component libraries.
- `@layer` lets you control cascade order explicitly — e.g., framework styles in one layer, your overrides in another that always wins.
- CSS nesting (native, no Sass needed) ships in all modern browsers.
- Logical properties: `margin-inline-start` instead of `margin-left` — for RTL support and future-proofing.

```
/* Container queries */
.card-wrapper { container-type: inline-size; }

.card { display: block; }
@container (min-width: 400px) {
  .card { display: flex; gap: 1rem; }
}

/* Cascade layers */
@layer reset, framework, components, overrides;

@layer framework {
  button { padding: 0.5rem 1rem; }
}
@layer overrides {
  button { padding: 0.75rem 1.5rem; } /* always wins */
}

/* Native nesting */
.card {
  padding: 1rem;
  & h2 { font-size: 1.25rem; }
  &:hover { background: #f4f4f8; }
}
```

Weekend 2 capstone

Pick a real product landing page (Linear, Stripe, Vercel, Notion). Rebuild it from scratch — semantic HTML, modern CSS, no framework. Must include: responsive layout (Grid + Flexbox), dark mode toggle, at least one `<dialog>`, accessible forms, and one container-query-responsive card. Push to GitHub. This artifact alone shows you can be trusted with frontend.

Week 3 — TypeScript

Goal: Read and write idiomatic TypeScript. Understand structural typing, generics, and the utility types you'll see everywhere in React code.

Day 11 — Basics, Type Inference, Structural Typing

Reading: TypeScript Handbook — [The Basics](#) + [Everyday Types](#).

Key insight: TS uses *structural* typing, not nominal. Two types are compatible if they have the same shape, regardless of name.

```
type Point = { x: number; y: number };
type Vector = { x: number; y: number };

const p: Point = { x: 1, y: 2 };
const v: Vector = p; // ✓ Same shape — works.

// Type inference is your friend — don't annotate what TS can figure out
const numbers = [1, 2, 3]; // inferred as number[]
const user = { name: 'Alice', age: 30 }; // inferred as { name: string; age: number }

// Annotate function parameters; let return types infer (mostly)
function greet(name: string) {
  return `Hello, ${name}`; // return type inferred as string
}
```

Exercise: Convert your Week 1 capstone CLI tool to TypeScript. Use `tsx` or `ts-node` to run it.

Day 12 — Unions, Intersections, Narrowing, Type Guards

Reading: TS Handbook — [Narrowing](#), then [Union Types](#).

```
type Status = 'idle' | 'loading' | 'success' | 'error';

type SuccessResponse = { status: 'success'; data: User };
type ErrorResponse = { status: 'error'; message: string };
type Response = SuccessResponse | ErrorResponse;

// Discriminated unions — TS narrows based on the literal
function handle(r: Response) {
  if (r.status === 'success') {
    console.log(r.data); // TS knows r is SuccessResponse here
  } else {
    console.log(r.message); // TS knows r is ErrorResponse here
  }
}

// Type guards
function isString(x: unknown): x is string {
  return typeof x === 'string';
}
```

Exercise: Model a state machine for a checkout flow (cart → shipping → payment → confirmation) using discriminated unions. Write functions that only accept specific states.

Day 13 — Generics

Reading: [TS Handbook — Generics](#).

Generics let functions and types work with any type while preserving type information.

```
// Without generics – loses type info
function firstAny(arr: any[]): any { return arr[0]; }

// With generics – preserves it
function first<T>(arr: T[]): T | undefined { return arr[0]; }
const n = first([1, 2, 3]);    // n: number | undefined
const s = first(['a', 'b']);   // s: string | undefined

// Constraints
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}

// Generic types
type ApiResponse<T> = {
  data: T;
  status: number;
  error?: string;
};

type UserResponse = ApiResponse<User>;
type PostsResponse = ApiResponse<Post[]>;
```

Exercise: Build a typed `fetchJson<T>` helper, a typed event emitter, and a typed cache class.

Day 14 — Utility Types, Mapped Types, Conditional Types

Reading: [TS Handbook — Utility Types](#), [Mapped Types](#), [Conditional Types](#).

These are what you'll see most in React libraries. Know them cold.

```
type User = { id: number; name: string; email: string; age: number };

type PartialUser = Partial<User>;           // all optional
type RequiredUser = Required<User>;         // all required
type PublicUser = Omit<User, 'email'>;      // remove email
type Credentials = Pick<User, 'email'>;     // keep only email
type UserKeys = keyof User;                 // 'id' | 'name' | 'email' | 'age'
type ReadonlyUser = Readonly<User>;         // can't mutate

// Mapped types (build your own)
type Nullable<T> = { [K in keyof T]: T[K] | null };

// Conditional types
type IsString<T> = T extends string ? true : false;
type A = IsString<'hello'>; // true
type B = IsString<42>;     // false
```

Day 15 — tsconfig, Modules, and Real-World Patterns

Reading: [TS tsconfig reference](#), [Modules](#), and Matt Pocock's [TSConfig Cheat Sheet](#) (free).

- Use `"strict": true` . Always. Non-negotiable.
- Know the common flags: `noUncheckedIndexedAccess` , `exactOptionalPropertyTypes` , `noImplicitOverride` .
- Use `unknown` over `any` . `any` turns off the type checker — `unknown` forces you to narrow.
- Prefer `type` for unions/aliases, `interface` for object shapes that may be extended. Both work most places.
- The `satisfies` operator (TS 4.9+) — typecheck a value against a type without widening it.

```
const config = {
  port: 3000,
  host: 'localhost',
  debug: true,
} satisfies Record<string, string | number | boolean>;

config.port.toFixed(2); // still typed as number – satisfies didn't widen it
```

Weekend 3 capstone

Build a typed REST API client library. Wrap `fetch` with generic methods (`get<T>` , `post<T, B>`), typed error handling, retries, and request/response interceptors. Publish it as a tiny npm package (even if private). You'll use a pattern just like this in real React apps.

Week 4 — React Fundamentals

Goal: Build small React apps with hooks, understand re-renders, and write idiomatic functional components.

Day 16 — Setup, JSX, Components, Props

Reading: [react.dev — Quick Start](#), [Your First Component](#), [Passing Props](#).

Setup: `npm create vite@latest my-app -- --template react-ts`. This gives you Vite + React + TS in 30 seconds.

```
type ButtonProps = {
  label: string;
  variant?: 'primary' | 'secondary';
  onClick?: () => void;
  children?: React.ReactNode;
};

export function Button({ label, variant = 'primary', onClick, children }: ButtonProps) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {children ?? label}
    </button>
  );
}
```

Mental model: A component is a function from props to UI. Re-rendering is React calling that function again. That's it.

Day 17 — useState, useEffect, the Re-render Model

Reading: [State: A Component's Memory](#), [Synchronizing with Effects](#), [You Might Not Need an Effect](#) (this one is crucial).

```
function Counter() {
  const [count, setCount] = useState(0);

  // Runs after every render where `count` changed
  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

Crucial rules:

- State updates are **asynchronous** and **batched**. `setCount(count + 1)` twice in a row only adds 1. Use the functional form: `setCount(c => c + 1)`.
- `useEffect`'s dependency array: omit nothing your effect uses. Use ESLint's `react-hooks/exhaustive-deps` rule.
- Effects clean up via the return function. Subscriptions, timers, listeners — always clean up.
- Don't use effects for things that aren't side effects. Computing derived state goes inline.

Exercise: Build a debounced search box. The input updates immediately, but the "search" API call only fires 300ms after the user stops typing. This forces you to use cleanup correctly.

Day 18 — Lists, Keys, Conditional Rendering, Forms

Reading: [Rendering Lists](#), [Conditional Rendering](#), [Reacting to Input with State](#).

```
function TodoList({ todos }: { todos: Todo[] }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          {todo.done ? <s>{todo.text}</s> : todo.text}
        </li>
      ))}
    </ul>
  );
}

// Controlled form
function NewTodo({ onAdd }: { onAdd: (text: string) => void }) {
  const [text, setText] = useState('');
  return (
    <form onSubmit={(e) => { e.preventDefault(); onAdd(text); setText(''); }}>
      <input value={text} onChange={e => setText(e.target.value)} />
      <button>Add</button>
    </form>
  );
}
```

Keys: Must be stable, unique among siblings, and ideally come from data (an ID). Never use array index unless the list never reorders.

Day 19 — Custom Hooks, Composition, Lifting State

Reading: [Reusing Logic with Custom Hooks](#), [Sharing State Between Components](#).

A custom hook is just a function whose name starts with `use` and which calls other hooks. They're how you share stateful logic.

```
function useLocalStorage<T>(key: string, initial: T) {
  const [value, setValue] = useState<T>(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initial;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}

// Usage
function App() {
  const [name, setName] = useLocalStorage('username', '');
  return <input value={name} onChange={e => setName(e.target.value)} />;
}
```

Lifting state: When two components need the same state, move it to their closest common ancestor.

Day 20 — Context, useReducer, useMemo, useCallback

Reading: [Passing Data Deeply with Context](#), [Extracting State Logic into a Reducer](#), [useMemo](#), [useCallback](#).

```
// useReducer – better than useState for complex state
type Action =
  | { type: 'add'; text: string }
  | { type: 'toggle'; id: string }
  | { type: 'remove'; id: string };

function todoReducer(state: Todo[], action: Action): Todo[] {
  switch (action.type) {
    case 'add': return [...state, { id: crypto.randomUUID(), text: action.text, done: false }];
    case 'toggle': return state.map(t => t.id === action.id ? { ...t, done: !t.done } : t);
    case 'remove': return state.filter(t => t.id !== action.id);
  }
}

// Context – for data needed deep in the tree
const ThemeContext = createContext<'light' | 'dark'>('light');
function useTheme() { return useContext(ThemeContext); }
```

When to memoize: Don't, until you measure. `useMemo` and `useCallback` have costs. React 19's compiler handles most cases automatically. Profile first.

Weekend 4 capstone

Build a Kanban board (Trello-lite). Three columns, draggable cards (use [@dnd-kit/core](#)), persisted in localStorage via your custom hook, with proper types. ~400 lines. This will surface every React pattern that matters.

Week 5 — React in the Real World

Goal: Data fetching, state management, routing, testing, and the ecosystem around React.

Day 21 — Data Fetching with TanStack Query

Reading: [TanStack Query — Overview](#) and [Quick Start](#).

`fetch` + `useEffect` is the wrong pattern for production. Use TanStack Query (formerly React Query) for server state.

```
import { useQuery } from '@tanstack/react-query';

function UserProfile({ id }: { id: string }) {
  const { data, isLoading, error } = useQuery({
    queryKey: ['user', id],
    queryFn: () => fetch(`/api/users/${id}`).then(r => r.json()),
  });

  if (isLoading) return <Spinner />;
  if (error) return <ErrorBanner />;
  return <div>{data.name}</div>;
}
```

You get caching, deduplication, background refetching, optimistic updates, and pagination — for free.

Day 22 — Client State: Zustand

Reading: [Zustand docs](#) (15 minutes — that's all it takes).

For global UI state (theme, modals, auth user), Redux is overkill. Zustand is ~1KB and has no boilerplate.

```
import { create } from 'zustand';

type AuthStore = {
  user: User | null;
  login: (user: User) => void;
  logout: () => void;
};

const useAuth = create<AuthStore>((set) => ({
  user: null,
  login: (user) => set({ user }),
  logout: () => set({ user: null }),
}));

// Usage
function Header() {
  const user = useAuth(s => s.user);
  return <div>{user?.name ?? 'Guest'}</div>;
}
```

Day 23 — Routing (TanStack Router or React Router)

Reading: [React Router docs](#) or [TanStack Router docs](#).

If you're going straight to Next.js after, routing in standalone React is mostly informational. Next.js handles routing differently (file-based).

Build one small SPA with React Router or TanStack Router so you understand the underlying primitives: route params, nested routes, navigation, loaders.

Day 24 — Testing

Reading: [Vitest docs](#), [React Testing Library](#), [Playwright](#).

Stack: Vitest + React Testing Library + Playwright.

```
import { render, screen } from '@testing-library/react';
import { userEvent } from '@testing-library/user-event';

test('clicking the button increments the count', async () => {
  render(<Counter />);
  const button = screen.getByRole('button', { name: /0/ });
  await userEvent.click(button);
  expect(screen.getByRole('button')).toHaveTextContent('1');
});
```

Rule: test behavior, not implementation. Query by accessible role/text, not by class names or test IDs.

Day 25 — Forms, Validation, Error Boundaries, Suspense

Reading: [React Hook Form — Get Started](#), [Zod docs](#), React's [Suspense](#).

For real forms: **React Hook Form** + **Zod** for validation.

```
import { z } from 'zod';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});

type FormData = z.infer<typeof schema>;

function LoginForm() {
  const { register, handleSubmit, formState: { errors } } =
    useForm<FormData>({ resolver: zodResolver(schema) });

  return (
    <form onSubmit={handleSubmit(data => console.log(data))}>
      <input {...register('email')} />
      {errors.email && <p>{errors.email.message}</p>}
      <input type="password" {...register('password')} />
      <button>Sign in</button>
    </form>
  );
}
```

Error boundaries catch render errors. Suspense suspends rendering for async data (paired with libraries that support it).

Weekend 5 capstone

Build a real app: GitHub repo search dashboard. Use TanStack Query to hit the GitHub API, Zustand for UI state, React Hook Form + Zod for the search form, and write tests for the core flow. Deploy to Vercel (free tier). This is portfolio-grade.

Week 6 — Next.js: The App Router

Goal: Understand Server Components, the App Router, data fetching, mutations, and the full-stack model.

Day 26 — Why Next.js, App Router Basics

Reading: [Next.js Learn — Dashboard App](#) (start here, work through it), [Next.js docs — Getting Started](#).

Setup: `npx create-next-app@latest --typescript --app --tailwind`.

- App Router is the current standard. Pages Router is legacy — ignore old tutorials.
- File-based routing: a folder under `app/` is a route. `app/dashboard/page.tsx` serves `/dashboard`.
- Special files: `layout.tsx`, `page.tsx`, `loading.tsx`, `error.tsx`, `not-found.tsx`.
- By default, components are **Server Components**. They run on the server, can be `async`, and can talk directly to databases.

```
// app/page.tsx — Server Component (default)
async function HomePage() {
  const posts = await fetch('https://api.example.com/posts').then(r => r.json());
  return (
    <ul>
      {posts.map((p: Post) => <li key={p.id}>{p.title}</li>)}
    </ul>
  );
}
export default HomePage;
```

Day 27 — Server vs Client Components

Reading: [Next.js — Server and Client Components](#), then [React docs — Server Components](#).

This is the single most important concept in modern Next.js.

- **Server Components** (default): run on the server. No `useState`, no `useEffect`, no event handlers. Can fetch data directly. Don't ship JS to the client.
- **Client Components**: opt-in with `'use client'` at the top of the file. Can use hooks and event handlers. Ship JS to the client.
- Rule: Server by default, Client when you need interactivity. Push `'use client'` as deep into the tree as possible.

```
// app/components/LikeButton.tsx
'use client';
import { useState } from 'react';

export function LikeButton() {
  const [liked, setLiked] = useState(false);
  return <button onClick={() => setLiked(!liked)}>{liked ? '♥' : '♡'}</button>;
}

// app/post/[id]/page.tsx — Server Component using a Client Component
import { LikeButton } from '@components/LikeButton';

export default async function Post({ params }: { params: { id: string } }) {
  const post = await getPost(params.id);
  return (
    <article>
      <h1>{post.title}</h1>
      <LikeButton />
    </article>
  );
}
```

Day 28 — Data Fetching, Caching, Revalidation

Reading: [Next.js — Fetching Data, Caching.](#)

Next.js extends `fetch` with caching options.

```
// Cached indefinitely (default)
const data = await fetch('https://api.example.com/data');

// Revalidate every 60 seconds
const data = await fetch('https://api.example.com/data', {
  next: { revalidate: 60 }
});

// Never cache
const data = await fetch('https://api.example.com/data', {
  cache: 'no-store'
});

// Tag-based revalidation (revalidate on demand)
const data = await fetch('https://api.example.com/posts', {
  next: { tags: ['posts'] }
});
// Later, from a Server Action: revalidateTag('posts');
```

Day 29 — Mutations with Server Actions

Reading: [Next.js — Updating Data with Server Actions.](#)

Server Actions are functions that run on the server but are called from the client. They replace most API routes.

```
// app/actions.ts
'use server';
import { revalidatePath } from 'next/cache';
import { db } from '@lib/db';

export async function createPost(formData: FormData) {
  const title = formData.get('title') as string;
  await db.post.create({ data: { title } });
  revalidatePath('/posts');
}

// app/posts/new/page.tsx
import { createPost } from '@app/actions';

export default function NewPost() {
  return (
    <form action={createPost}>
      <input name="title" />
      <button>Create</button>
    </form>
  );
}
```

Day 30 — Layouts, Loading, Error Handling, Metadata

Reading: [Layouts and Pages](#), [Error Handling](#), [Metadata](#).

```
// app/dashboard/layout.tsx – wraps all dashboard pages
export default function DashboardLayout({ children }: { children: React.ReactNode }) {
  return (
    <div className="grid grid-cols-[200px_1fr]">
      <Sidebar />
      <main>{children}</main>
    </div>
  );
}

// app/dashboard/loading.tsx – shown while page.tsx is loading
export default function Loading() { return <Spinner />; }

// app/dashboard/error.tsx – shown if page.tsx throws
'use client';
export default function Error({ error, reset }: { error: Error; reset: () => void }) {
  return <><p>{error.message}</p><button onClick={reset}>Retry</button></>;
}

// Metadata
export const metadata = { title: 'Dashboard', description: 'User dashboard' };
```

Weekend 6 capstone

Build a blog: list page (Server Component, fetched at build time), individual post pages (dynamic routes), an admin page to create posts (Server Action), with proper loading/error states. Use a free Postgres (Neon or Supabase) and Prisma or Drizzle as the ORM.

Week 7 — Next.js: Production

Goal: Authentication, middleware, deployment, performance, SEO, and the full production checklist.

Day 31 — Authentication with Auth.js (NextAuth v5)

Reading: [Auth.js — Get Started with Next.js](#).

The standard for auth in Next.js. Supports OAuth, magic links, credentials.

```
// auth.ts
import NextAuth from 'next-auth';
import GitHub from 'next-auth/providers/github';

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [GitHub],
});

// app/api/auth/[...nextauth]/route.ts
export { GET, POST } from '@auth';

// In a Server Component
import { auth } from '@auth';

export default async function Page() {
  const session = await auth();
  if (!session) return <p>Please sign in</p>;
  return <p>Hello, {session.user?.name}</p>;
}
```

Day 32 — Middleware, Route Handlers, Edge Runtime

Reading: [Next.js — Middleware](#), [Route Handlers](#).

```
// middleware.ts — runs before every matching request
import { NextResponse } from 'next/server';
import { auth } from '@auth';

export async function middleware(request: Request) {
  const session = await auth();
  if (!session && request.url.includes('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
  return NextResponse.next();
}

export const config = { matcher: ['/dashboard/:path*'] };

// app/api/posts/route.ts — REST endpoint
import { NextResponse } from 'next/server';

export async function GET() {
  const posts = await db.post.findMany();
  return NextResponse.json(posts);
}

export async function POST(request: Request) {
  const body = await request.json();
  const post = await db.post.create({ data: body });
  return NextResponse.json(post, { status: 201 });
}
```

Day 33 — Performance: Streaming, Suspense, Images, Fonts

Reading: [next/image](#), [next/font](#), [Streaming / PPR](#).

- Wrap slow Server Components in `<Suspense>` to stream them in — the rest of the page renders immediately.
- Use `next/image` for automatic responsive images, lazy loading, AVIF/WebP.
- Use `next/font` for zero-layout-shift, self-hosted fonts.
- Use `next/dynamic` for code-splitting client components that aren't immediately visible.

```
import { Suspense } from 'react';
import Image from 'next/image';
import { Inter } from 'next/font/google';

const inter = Inter({ subsets: ['latin'] });

export default function Page() {
  return (
    <div className={inter.className}>
      <Image src="/hero.jpg" alt="Hero" width={1200} height={600} priority />
      <Suspense fallback=<Skeleton />>
        <SlowComponent />
      </Suspense>
    </div>
  );
}
```

Day 34 — SEO, Metadata, Sitemaps, Open Graph

Reading: [generateMetadata](#), [sitemap.ts](#), [Open Graph images](#).

```
// app/blog/[slug]/page.tsx
import type { Metadata } from 'next';

export async function generateMetadata({ params }: { params: { slug: string } }): Promise<Metadata> {
  const post = await getPost(params.slug);
  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      images: [{ url: post.coverImage }],
    },
  };
}

// app/sitemap.ts
export default async function sitemap() {
  const posts = await db.post.findMany();
  return [
    { url: 'https://example.com', lastModified: new Date() },
    ...posts.map(p => ({ url: `https://example.com/blog/${p.slug}`, lastModified: p.updatedAt })),
  ];
}
```

Day 35 — Deployment, Observability, the Production Checklist

Reading: [Next.js — Deploying, Vercel docs.](#)

Deploy: [Vercel](#) (one click from a GitHub repo). For non-Vercel, Next.js runs anywhere Node runs.

Production checklist:

- Environment variables in `.env.local`, never committed. Use `process.env.X` with a Zod schema to validate at boot.
- Error tracking: Sentry (free tier).
- Analytics: Vercel Analytics, Plausible, or PostHog.
- Database: connection pooling (Neon and Supabase handle this; bare Postgres needs PgBouncer).
- Caching strategy documented per route. Know what's static, what's dynamic, what revalidates.
- Rate limiting on Server Actions and API routes (Upstash Redis is the standard).
- Bundle analysis: [@next/bundle-analyzer](#). Anything over 200KB gzipped on a page needs justification.
- Accessibility audit: Lighthouse score > 95 for accessibility on every page.

Week 7 capstone — the real one

Build a SaaS-grade app end to end: auth, billing (Stripe), database, dashboard, public marketing pages, blog, transactional emails (Resend), and deployment. Pick a small but real idea — link shortener, habit tracker, invoice generator, whatever. Ship it to production. This is the artifact that gets you hired or starts your product.

Appendix — Reference & Resources

Essential free resources (bookmark all)

Topic	Resource	URL
Modern JS	Exploring JavaScript	exploringjs.com/js
JS deep dive	You Don't Know JS Yet	github.com/getify/You-Dont-Know-JS
HTML / CSS / Web APIs	MDN Web Docs	developer.mozilla.org
CSS course	web.dev Learn CSS	web.dev/learn/css
Flexbox / Grid	CSS Tricks complete guides	css-tricks.com
TypeScript	TS Handbook	typescriptlang.org/docs/handbook
React	Official React docs	react.dev
Next.js	Official Next.js Learn	nextjs.org/learn
Patterns	Patterns.dev	patterns.dev

Toolchain you'll actually use

Purpose	Tool
Bundler / dev server	Vite (standalone) or Next.js (full-stack)
Package manager	pnpm (faster, disk-efficient)
Linting / formatting	ESLint + Prettier (or Biome)
Styling	Tailwind CSS
Component primitives	shadcn/ui (copy-paste, not a dependency)
Server state	TanStack Query
Client state	Zustand
Forms + validation	React Hook Form + Zod
ORM	Drizzle (TypeScript-first) or Prisma
Auth	Auth.js (NextAuth v5) or Clerk
Testing	Vitest + React Testing Library + Playwright
Deployment	Vercel (or Cloudflare , Netlify , fly.io)

Things to consciously skip

- jQuery, Bootstrap, Webpack tutorials, Gulp, Grunt — historical at this point.
- Redux — only if a job needs it. Most new code doesn't.
- Class components — you'll see them in old codebases, that's it.
- Pages Router for Next.js — App Router is the present and future.
- CSS-in-JS (styled-components, Emotion) — slowly being replaced by Tailwind + CSS Modules. Learn it only if a job uses it.

The mindset that makes this work

You have 25 years of intuition about programs. Don't let "I should know this" embarrass you when CSS Grid clicks on day 9 instead of day 1, or when Server Components make you re-learn what a component "is." The pace here is fast

precisely because you already know about state, async, abstractions, and trade-offs. You're not learning to program. You're learning where modern frontend chose to put the complexity.

— End of curriculum —